UCS503 – Software Engineering

ClassLink

A Cross-Platform BLE Proximity System for Classroom Attendance

Submitted by:

1024160102 Mahi Singh

1024160109 Vrishank Nehru

1024160123 Khagendra Saini

1024160111 Vemula Manvi Smaran

Group: 3P14 BE Third Year, CSE

Submitted to:

Miss Anushka

Mid-Semester Evaluation

Computer Science and Engineering Department

Thapar Institute of Engineering and Technology, Patiala

Contents

1 Introduction **2**
- 1.1 Background and Context . . . 2
- 1.2 Problem Statement . . . 2
- 1.3 Project Objectives . . . 2

2 Methodology and Design **3**
- 2.1 System Architecture . . . 3
  - 2.1.1 High-Level Design . . . 3
  - 2.1.2 Technical Stack . . . 3
- 2.2 System Design Diagrams . . . 3
  - 2.2.1 Use Case Diagram . . . 4
  - 2.2.2 Sequence Diagram . . . 5
  - 2.2.3 Class Diagram . . . 5
  - 2.2.4 Component / Architecture Diagram . . . 6
  - 2.2.5 Data Flow Diagram . . . 6
- 2.3 Implementation Details . . . 7
  - 2.3.1 Module 1: Cross-Platform BLE Advertising . . . 7
  - 2.3.2 Module 2: Android 12+ Permission Handling . . . 8
  - 2.3.3 Scope Boundaries . . . 8

3 Results and Evaluation **10**
- 3.1 Testing Methodology . . . 10
- 3.2 Performance Metrics . . . 10
- 3.3 Analysis . . . 11

4 Conclusion **12**
- 4.1 Summary of Work . . . 12
- 4.2 Key Findings . . . 12
- 4.3 Future Work and Recommendations . . . 12
- 4.4 Final Remarks . . . 13

Chapter 1

Introduction

1.1 Background and Context

Manual attendance in a classroom is slow, easy to falsify, and produces no verifiable record of who was physically present. ClassLink addresses this by using Bluetooth Low Energy (BLE) proximity as the basis for attendance check-in: a teacher's device advertises a short-lived session code, and only student devices that are physically nearby can detect that signal and check in. This report documents the completed BLE discovery and proximity vertical slice of ClassLink, built as Phase 4 of a larger, contractually scoped system.

1.2 Problem Statement

The core technical problem is establishing reliable, cross-platform proximity-based discovery between a teacher device and multiple student devices using BLE alone, without relying on a backend, while working around platform-specific restrictions in how Android and iOS expose BLE advertisement data. A secondary problem is ensuring that proximity decisions are robust to noisy RSSI readings rather than reacting to a single stray packet.

1.3 Project Objectives

- **Objective 1:** Implement a working BLE peripheral/central architecture in Flutter, where a teacher device advertises a session and student devices discover it directly, with no intermediate mesh or relay.
- **Objective 2:** Design a session-code transport that works identically from the student scanner's perspective on both Android and iOS, despite each platform exposing different subsets of BLE advertisement fields.
- **Objective 3:** Validate the proximity check against noisy real-world RSSI behaviour using a windowed, median-based threshold rather than a single-sample decision, and verify the implementation with unit tests and real-device builds.

Chapter 2

Methodology and Design

2.1 System Architecture

ClassLink's BLE vertical slice follows a direct peripheral–central model with no mesh or relay hop, matching the project's "no BLE mesh in MVP" design constraint.

2.1.1 High-Level Design

- **Teacher device (BLE peripheral / advertiser):** starts a session, generates a 6-digit code, and broadcasts it over BLE.
- **Student device (BLE central / scanner):** scans for the ClassLink service UUID, collects RSSI samples over a fixed window, and once a valid proximity signal is confirmed, connects briefly to write a check-in payload containing the student's name and roll number.
- **Proximity decision:** the median of RSSI samples over a 4-second window is compared against a -75 dBm threshold, so a single noisy reading cannot trigger a false check-in.

2.1.2 Technical Stack

- **Framework/Language:** Flutter/Dart, using native BLE peripheral and central APIs on Android and iOS.
- **Database:** None in this phase (see Section 2.3.3 on scope boundaries).
- **Tools:** `flutter analyze` for static analysis, Flutter's unit test framework, Apple's `devicectl` for physical iPhone deployment, and a Flutter release build for sideloaded Android APK distribution.
- **Libraries:** Platform BLE bindings for CoreBluetooth (iOS) and the Android Bluetooth LE stack.

2.2 System Design Diagrams

This section presents the system design using standard UML and structured-analysis views: a Use Case diagram showing the two actors and their interactions with the system, a Sequence diagram tracing the BLE advertising and check-in flow over time, a Class

diagram of the core BLE components, a Component/Architecture diagram showing the current on-device scope alongside the planned backend, and a Data Flow Diagram (Level 0 and Level 1) showing how data moves through the system.

2.2.1 Use Case Diagram

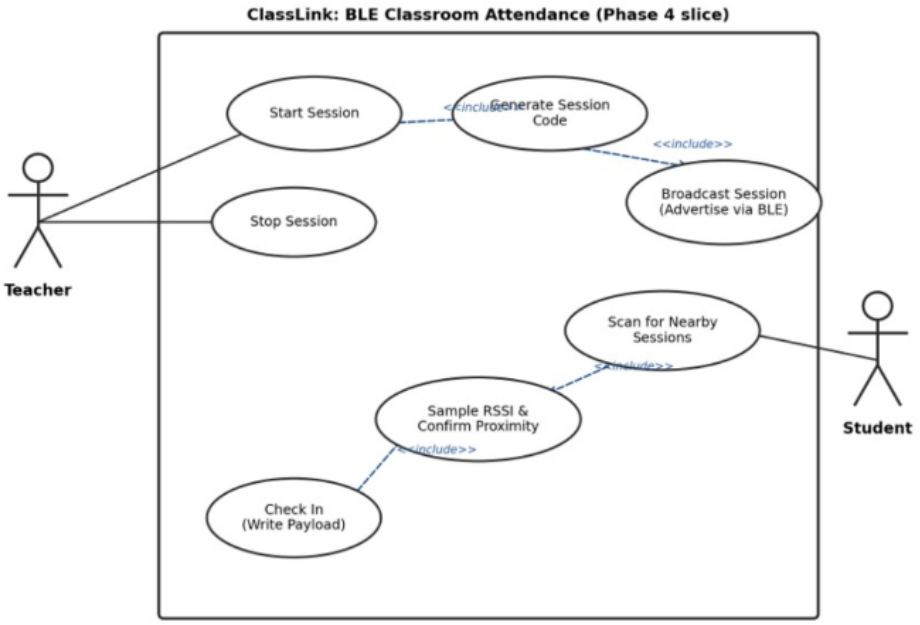


Figure 2.1: Use Case Diagram for ClassLink

2.2.2 Sequence Diagram

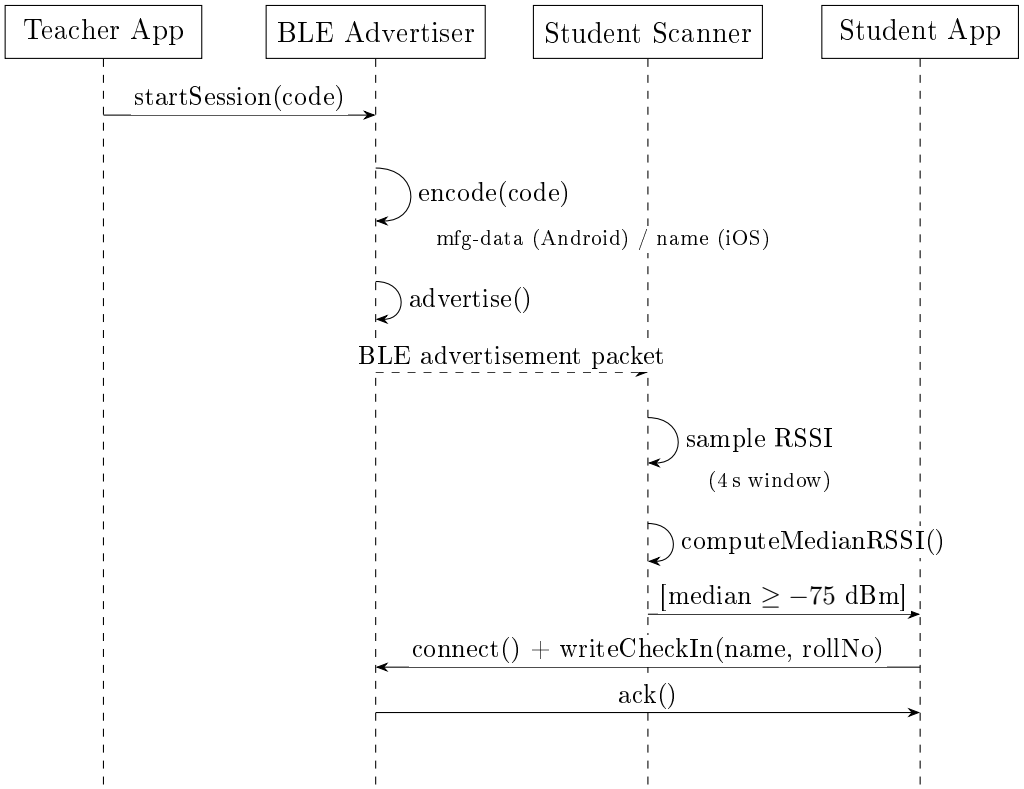


Figure 2.2: Sequence Diagram: BLE Advertising and Check-in Flow

2.2.3 Class Diagram

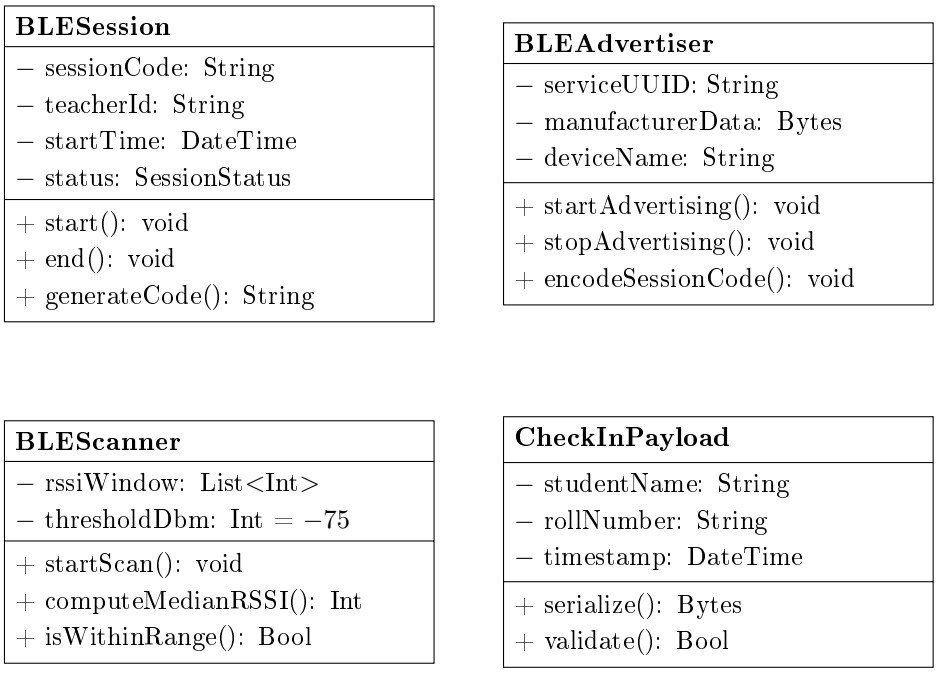


Figure 2.3: Class Diagram: Core BLE Components

2.2.4 Component / Architecture Diagram

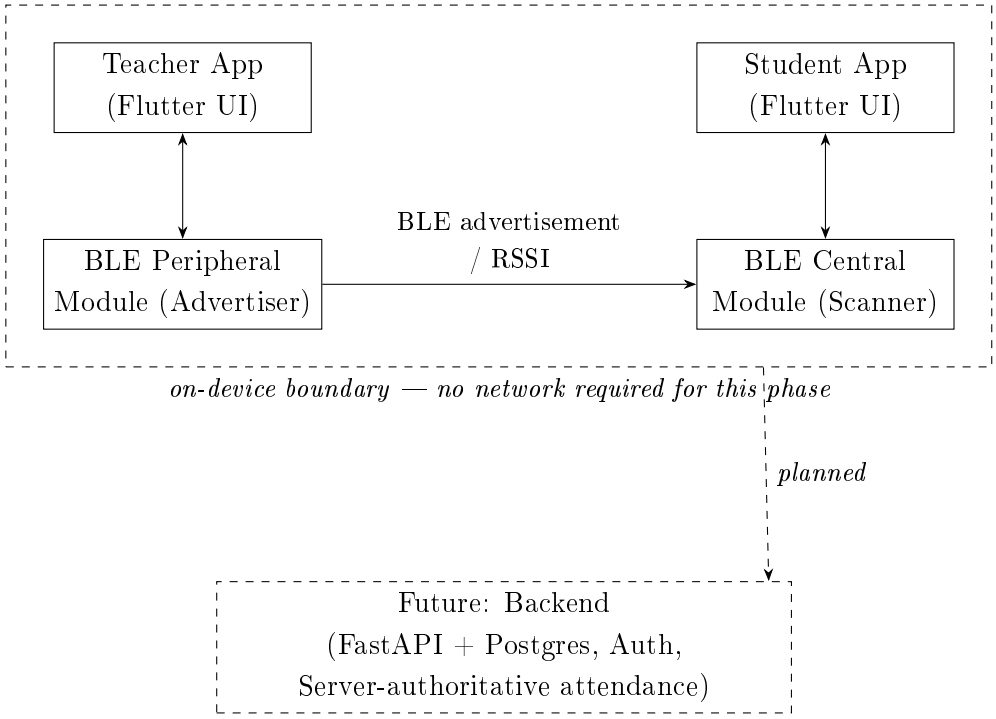


Figure 2.4: High-Level Component Architecture

2.2.5 Data Flow Diagram

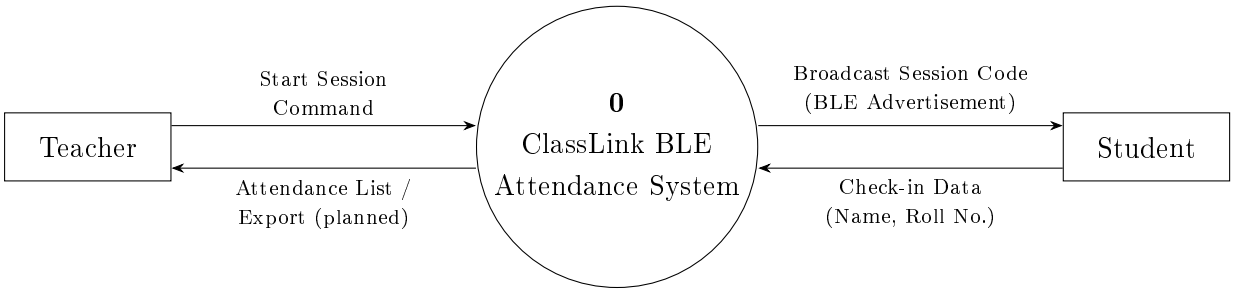


Figure 2.5: Data Flow Diagram — Level 0 (Context Diagram)

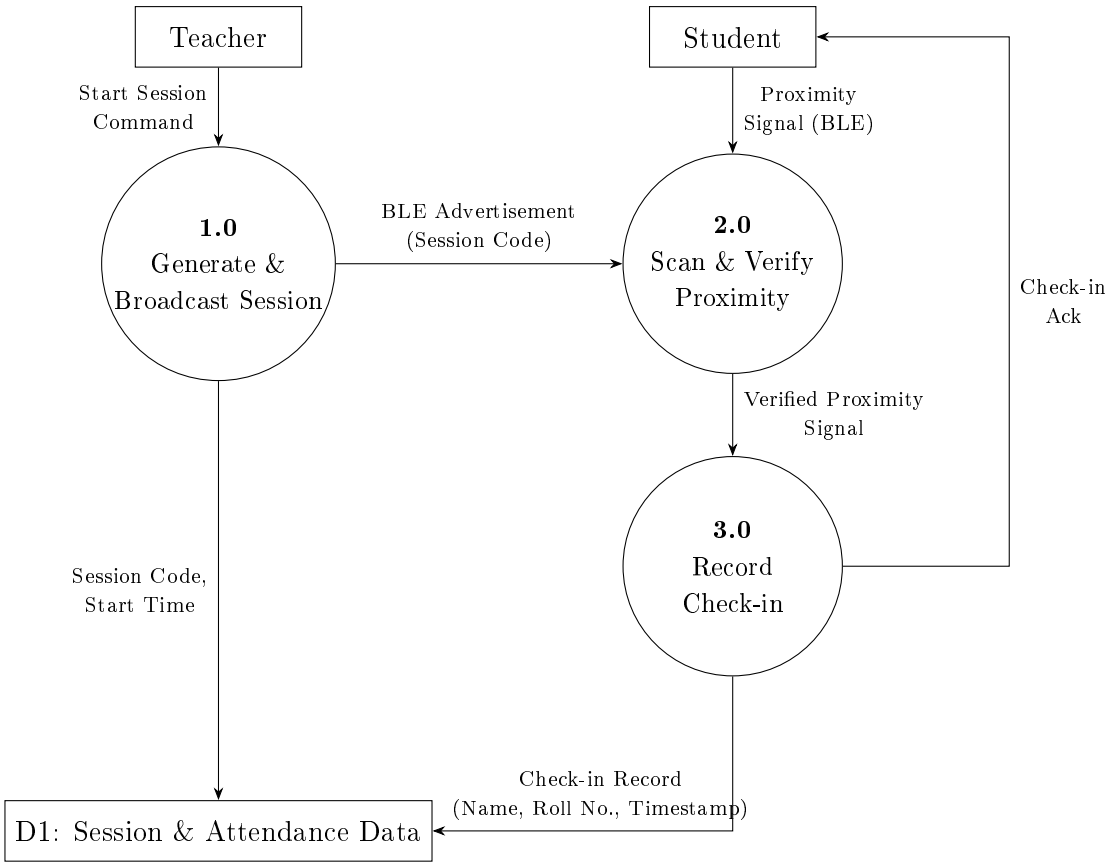


Figure 2.6: Data Flow Diagram — Level 1

2.3 Implementation Details

2.3.1 Module 1: Cross-Platform BLE Advertising

The central engineering problem solved in this phase was cross-platform BLE advertising. iOS's CoreBluetooth framework silently drops manufacturer-specific data from BLE advertisements when the app is in the background or, in practice, restricts advertisement content to a local device name and service UUIDs only; Android has no equivalent restriction. Because the original design encoded the session code as manufacturer-specific data, it worked on Android but silently failed on iOS with no runtime error, making the defect difficult to detect without cross-platform testing.

The fix was a platform-adaptive transport:

- **Android:** the session code is sent as 10 bytes of manufacturer-specific data alongside the service UUID. This fits the legacy 31-byte BLE advertisement limit exactly: 3 bytes of flags, 18 bytes for the 128-bit service UUID, and 10 bytes of manufacturer data.
- **iOS:** the session code is instead embedded in the advertised local device name, formatted as `CL<6-digit-code>` (e.g. `CL042917`). This is deliberately not used on Android, since setting the advertised name there renames the phone's actual

Bluetooth adapter.

- **Scanner logic:** the student-side scanner checks manufacturer data first, then falls back to parsing the advertised device name, so a single scanner implementation can discover either an Android or an iOS teacher device without knowing in advance which platform is advertising.

Two further defects were identified and fixed during this work. First, the original advertisement omitted the service UUID that the scanner itself filtered on, which made discovery impossible on both platforms. Second, BLE commands were being issued before the Bluetooth stack reported a `poweredOn` state, which CoreBluetooth silently rejects on a fresh app launch; this was fixed with a wait-and-timeout guard that surfaces specific, actionable errors (Bluetooth off, unauthorized, or unsupported) instead of failing silently.

2.3.2 Module 2: Android 12+ Permission Handling

A second defect was found in how the app requested runtime permissions on Android. The app originally requested `ACCESS_FINE_LOCATION` as a hard requirement on all Android versions in order to perform BLE scanning. On Android 12 and above, the manifest correctly declares `BLUETOOTH_SCAN` with the `neverForLocation` flag, since the app only uses RSSI for proximity and never derives physical location. However, that flag also causes the OS to auto-deny any accompanying location permission request, which would have silently blocked the entire check-in flow on any modern Android phone. This was fixed by treating Bluetooth permissions as the hard requirement on Android 12+, and location permission as a best-effort request that is only relevant on pre-Android 12 devices.

2.3.3 Scope Boundaries

This proof-of-concept intentionally implements only Phase 4 (the BLE vertical slice) of a larger system defined in the project's own engineering contract. The following are deliberately out of scope for this phase, and are noted here so the evaluation scope is unambiguous:

- No backend (FastAPI) and no database (Supabase/Postgres); no Row-Level Security.
- No authentication, no roles, and no classroom/enrollment model.
- No rotating session tokens, no server-issued challenges, and no cryptographic device binding or signing.
- No biometric authentication step.
- Attendance decisions currently happen client-side; the project's contract requires all attendance mutation to be server-authoritative in the final system. This is a known, tracked gap rather than an oversight.

- No audit logging and no manual-correction workflow. CSV export UI exists on-screen but is not yet wired to a real backend.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

- **Unit Testing:** 16 unit tests were written and pass, including 5 tests added specifically to cover the session-code transport: manufacturer-data round-trip, iOS name round-trip, rejection of malformed or unrelated advertised names, and correctness of the 31-byte advertisement budget.
- **Static Analysis:** `flutter analyze` reports no issues on the current codebase.
- **Real-Device Testing:** the app was installed and launched on a physical iPhone using a release build via `devicectl` (chosen because wireless `flutter run` debugging proved unreliable), and a real release APK (45 MB, containing compiled `classes.dex`) was built and verified for Android.
- **Cross-Platform Testing:** a two-phone cross-platform test (iPhone-as-teacher with Android-as-student, and the reverse role assignment) is the next concrete milestone and had not been formally executed and recorded at the time of writing; this is flagged here as the immediate next validation step, to be captured with timestamped screenshots.

3.2 Performance Metrics

Metric	**Result**
Unit tests passing	16 / 16
Static analysis issues	0
Proximity window	4 s (median RSSI)
Proximity threshold	−75 dBm
Android advertisement budget used	31 / 31 bytes
Android release APK size	45 MB

Table 3.1: Testing and Build Results Summary

3.3 Analysis

The primary objective of this phase — proving that a single BLE scanner implementation can reliably discover both an Android and an iOS teacher device — was met.

The iOS/CoreBluetooth advertisement-data restriction was not documented clearly enough in the BLE library's own documentation to anticipate in advance; it was discovered empirically by observing that discovery worked on Android but failed silently on iOS, which is itself a notable finding for a BLE-based system built across both mobile platforms. The windowed, median-based proximity check was chosen specifically to avoid false check-ins from single noisy RSSI samples, and the Android 12+ permission fix removed a defect that would otherwise have silently blocked check-in on the majority of current Android devices.

The main outstanding challenge is that the two-phone, cross-platform, real-world check-in has not yet been formally executed and recorded, even though each platform has been independently verified in isolation (unit tests, static analysis, and single-device real hardware runs). This is the immediate next step before this vertical slice can be considered fully validated end-to-end.

Chapter 4

Conclusion

4.1 Summary of Work

This report presented ClassLink's completed BLE discovery and proximity vertical slice: a teacher device advertises a session over BLE, and student devices discover it and check in based on a windowed proximity check, using a session-code transport that adapts to each platform's BLE advertisement constraints. This represents Phase 4 of a larger planned system and was validated through unit testing, static analysis, and real-device builds on both Android and iOS.

4.2 Key Findings

- **Finding 1:** iOS's CoreBluetooth framework restricts BLE advertisement content to a local device name and service UUIDs, silently dropping manufacturer-specific data; this required a platform-adaptive session-code transport rather than a single shared encoding.
- **Finding 2:** Android's `neverForLocation` flag on `BLUETOOTH_SCAN`, while correct for an app that only uses RSSI proximity, causes the OS to auto-deny any accompanying location permission request on Android 12+, which would have silently blocked the entire check-in flow if location had remained a hard requirement.
- **Finding 3:** a single, unified scanner implementation can support discovery of either platform's teacher device by checking manufacturer data first and falling back to parsing the advertised device name, avoiding the need for platform-specific scanner logic on the student side.

4.3 Future Work and Recommendations

- Execute and formally record the two-phone, cross-platform check-in test (iPhone teacher with Android student, and the reverse), with timestamped screenshots.
- Build the server-authoritative backend (FastAPI with a Supabase/Postgres database and Row-Level Security) so attendance mutation no longer happens client-side.
- Add authentication, roles, and a classroom/enrollment model.

- Introduce rotating session tokens, server-issued challenges, and cryptographic device binding or signing, in place of the current static 6-digit code.
- Add a biometric authentication step, audit logging, a manual-correction workflow, and wire the existing CSV export UI to a real backend.

4.4 Final Remarks

This phase deliberately targeted the hardest and most platform-fragile part of ClassLink — BLE discovery and proximity detection under real operating-system constraints — before investing in the server-authoritative security model that the project's engineering contract ultimately requires. With that vertical slice working and tested, the project is well positioned to move on to the backend, authentication, and security work that remains.